Concurrency Control & Crash Recovery

Mr. S.K.Patil

Concurrency Control & Crash Recovery

Introduction –

- A **collection of several operations** on the database **appears to be a single unit** from the **point of view of the database user.**
- For **example**, a **transfer of funds** from a **checking account to a savings account** is a **single operation** from the customer's point of view.
- **But** within the **database system** it **consists of several operations.**
- It is **essential** that **all these operations occur**, or **in case of a failure, none occur**.
- **Transactions - Collections of operations that form a single logical unit of work.**
- A **database system** must **ensure proper execution of transactions despite failures** - **either** the **entire transaction executes**, or **none of it does**.
- Database system must **manage concurrent execution of transactions** in such a way that it **avoids the inconsistency.**

Concurrency Control & Crash Recovery

Transaction Concept –

- A **transaction** is a **unit of program execution** that **accesses and possibly updates various data items.**
- A **transaction** is **initiated by a user program written in a high-level data-manipulation language**(typically SQL), **or programming language** (for example, C++, or Java).
- The **transaction consists of all operations executed between** the **begin transaction** and **end transaction**.
- This **collection of steps** must **appear to the user as a single, indivisible unit.**
- Since a **transaction is indivisible**, it **either executes** in its **entirety or not at all.**
- Thus, if a **transaction begins to execute but fails** for whatever reason, **any changes to the database** that the transaction may have made **must be undone.**

Concurrency Control & Crash Recovery

ACID Properties –

- To ensure integrity of the data, it is required that the database system maintain the following properties of the transactions:
  - **A**tomicity
  - **C**onsistency
  - **I**solation
  - **D**urability
- These properties are called the **ACID** properties.
- **Atomicity -** Either all operations of the transaction are reflected properly in the database, or none are.
- **Consistency -** Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.

Concurrency Control & Crash Recovery

ACID Properties –

- **Isolation -** Even though **multiple transactions may execute concurrently**, the **system guarantees that**, for **every pair of transactions Ti and Tj**, it appears to Ti that **either Tj finished execution before Ti started or Tj started execution after Ti finished.** Thus, **each transaction is unaware of other transactions executing concurrently in the system.**
- **Durability - After a transaction completes successfully**, the **changes** it has **made to** the **database persist**, **even if** there are **system failures.**

Concurrency Control & Crash Recovery

Transaction State –

- In the **absence of failures**, **all transactions** must **complete successfully.**
- A **transaction may not always complete its execution successfully**. Such a transaction is termed as **aborted.**
- To **ensure the atomicity property**, an **aborted transaction** must have **no effect on the state of the database.**
- **Any changes** that the **aborted transaction made to the database must be undone.**
- Once the **changes caused by an aborted transaction have been undone**, we say that the **transaction** has been **rolled back.**
- It is part of the **responsibility of the recovery scheme to manage transaction aborts.** This is done typically by **maintaining a log.**

Concurrency Control & Crash Recovery

Transaction State –

- A **transaction** that **completes its execution successfully** is said to be **committed.**
- A **committed transaction** that has **performed updates transforms the database into a new consistent state**, which must **persist even if** there is a **system failure**.
- Once a **transaction has committed**, we **cannot undo its effects** by aborting it.
- The **only way to undo the effects** of a committed transaction is to **execute a compensating transaction.**
- For instance, if a transaction added $20 to an account, the compensating transaction would subtract $20 from the account.

Concurrency Control & Crash Recovery

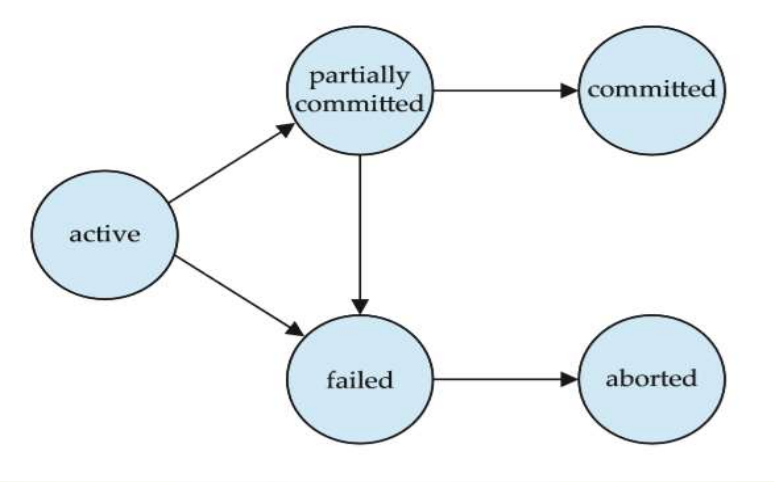
Simple Transaction Model –

- For a successful completion of transaction, a transaction must be in one of the following states:
- **Active -** the initial state; the transaction stays in this state while it is executing
- **Partially committed** - after the final statement has been executed
- **Failed** - after the discovery that normal execution can no longer proceed
- **Aborted** - after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction
- **Two options after** it has been **aborted**:
  - **Restart the transaction** - only if no internal logical error
  - **Kill the transaction**
- **Committed - after successful completion**

Concurrency Control & Crash Recovery

Simple Transaction Model –

- The **state diagram** corresponding to a transaction appears in Figure –



- We say that a **transaction has committed only if** it has **entered the committed state.**
- Similarly, we say that a **transaction has aborted only if** it has **entered the aborted state.**
- A **transaction** is said to have **terminated if** has **either committed or aborted**.

Concurrency Control & Crash Recovery

Simple Transaction Model –

- Consider the transaction concept in a simple bank application consisting of several accounts and a set of transactions that access and update those accounts.
- **Transactions access data using two operations**:
  - **read(X)**, which **transfers the data item X from the database to a variable**, also called X, in a buffer in main memory belonging to the transaction that executed the read operation.
  - **write(X)**, which **transfers the value in the variable X in the main-memory buffer of the transaction** that executed the write **to the data item X in the database.**

Concurrency Control & Crash Recovery

Simple Transaction Model –

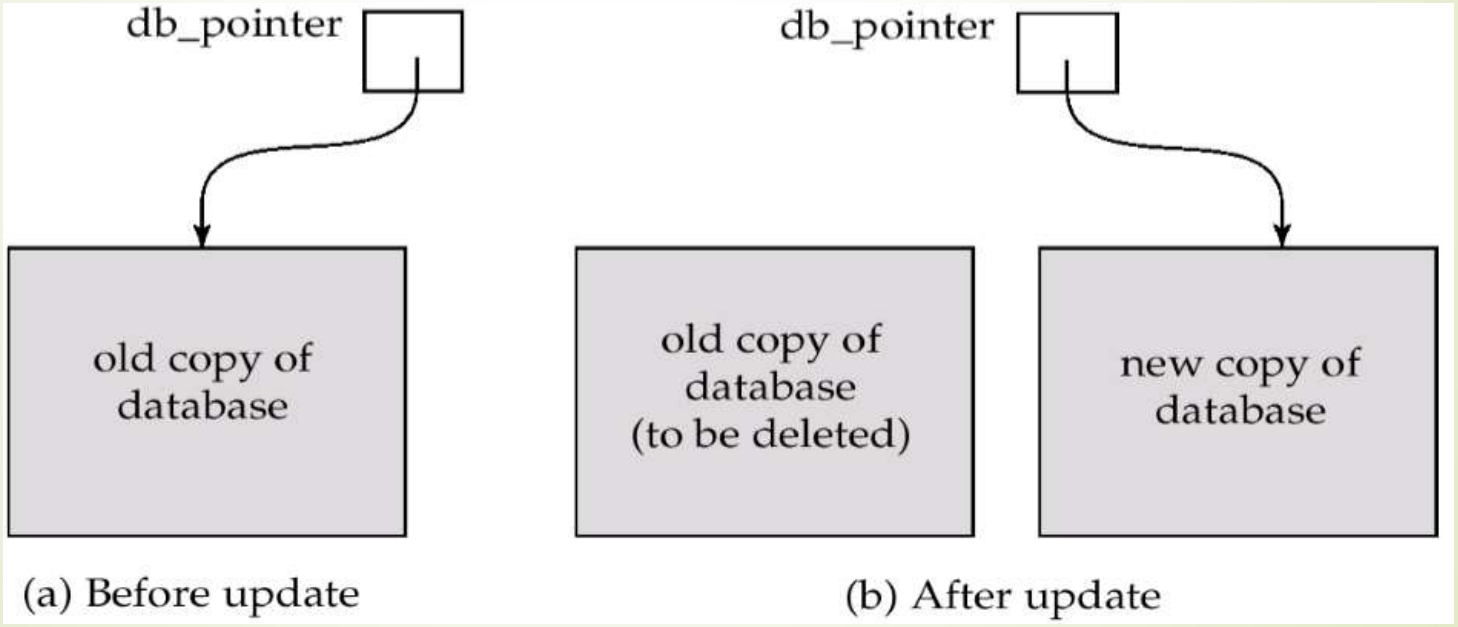
- Let Ti be a transaction that transfers $50 from account A to account B. This transaction can be defined as:

T_i: read(A);
$A := A - 50$;
write(A);
read(B);
$B := B + 50$;
write(B).

Concurrency Control & Crash Recovery

Implementation of Atomicity and Durability –

- The **recovery-management component of a database** system implements the **support for atomicity and durability.**
- **Shadow Database Scheme** -



Concurrency Control & Crash Recovery

Implementation of Atomicity and Durability –

- **Shadow Database Scheme** –
  - Assume that **only one transaction is active** at a time.
  - A pointer called **db_pointer** always **points to the current consistent copy of the database.**
  - All **updates are made** on a **shadow copy** of the database.
  - A **db_pointer** is made to **point to** the **updated shadow copy only after** the **transaction reaches partial commit** and **all updated pages have been flushed to disk.**
  - **In case transaction fails**, **old consistent copy pointed by db_pointer can be used**, and the **shadow copy** can be **deleted.**

Concurrency Control & Crash Recovery

Concurrent Executions –

- **Multiple transactions** are allowed to **run simultaneously** in the system.
- **Advantages –**
  - **Increased processor and disk utilization –**
    - Leading to **better transaction throughput** i.e. **one transaction** can be **using the CPU** while **another is reading from or writing to the disk.**
  - **Reduced average response time for transactions –**
    - **Short transactions need not wait behind long ones**
  - **Concurrency control schemes –**
    - **Mechanisms to achieve isolation**, i.e. to **control the interaction among the concurrent transactions** in order **to prevent** them from destroying the **consistency of the database**

Concurrency Control & Crash Recovery

Schedules –

- **Schedules** are the **sequences** that **indicate** the **chronological order** in which **instructions of concurrent transactions are executed**.
- A **schedule** for a set of transactions must **consist of all instructions** of those transactions.
- A **schedule must preserve** the **order** in which the instructions appear in each individual transaction.
- **Schedules in DBMS** are a **series of operations performing one transaction to the other.**
- **R(X)** means **Reading the value: X**; and **W(X)** means **Writing the value: X.**
- **Schedules** in DBMS are of **two types** –
  - **Serial Schedule**
  - **Non-serial Schedule**

Concurrency Control & Crash Recovery

Schedules –

- **Serial Schedule –**
  - A **schedule** in which **only one transaction is executed at a time** i.e. one transaction is executed completely before starting another transaction.
  - **Serial schedules** are **always serializable** because the **transactions only work one after the other**. For a transaction, there are n! serial schedules possible. (n - no. of transactions).
  - Here, we can see that Transaction-2 starts its execution after the completion of Transaction-1.

Transaction – 1	Transaction – 2
R(a)	
W(a)	
R(b)	
W(b)	
	R(b)
	W(b)
	R(a)
	W(a)

Concurrency Control & Crash Recovery

Schedules –

- **Non-Serial Schedule –**
  - A **schedule** in which the **transactions are interleaving or interchanging.**
  - There are several transactions executing simultaneously as they are being used in performing real-world database operations.
  - These transactions may be working on the same piece of data.
  - Hence, the serializability of non-serial schedules is a major concern so that our database is consistent before and after the execution of the transactions.

Concurrency Control & Crash Recovery

Schedules –

- **Non-Serial Schedule –**
  - We can see that Transaction-2 starts its execution before the completion of Transaction-1, and they are interchangeably working on the same data, i.e., "a" and "b".

Transaction – 1	Transaction – 2
R(a)	
W(a)	
	R(b)
	W(b)
R(b)	
	R(a)
W(b)	
	W(a)

Concurrency Control & Crash Recovery

Schedules –

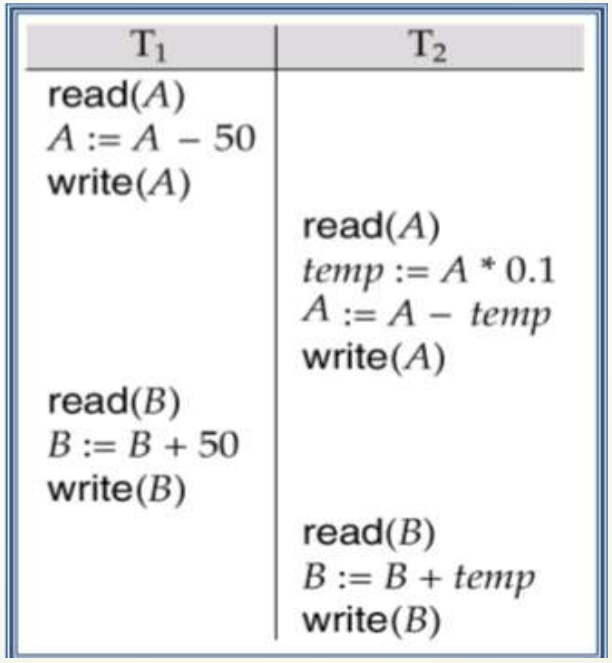
- **Example** - Let T1 transfer $50 from A to B, and T2 transfer 10% of the balance from A to B. The following is a **serial schedule** in which **T1 is followed by T2**.

T_1	T_2
read(A)	
$A := A - 50$	
write (A)	
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)

Concurrency Control & Crash Recovery

Schedules –

- **Example** - Let T1 and T2 be the transactions defined previously. The following schedule is **not a serial schedule**, but it is **equivalent to Schedule 1.**



T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
read(B)	
$B := B + 50$	
write(B)	
	read(B)
	$B := B + temp$
	write(B)

- **In both Schedule**, the **sum A + B is preserved.**

Concurrency Control & Crash Recovery

Schedules –

- **Example** - The following concurrent **schedule does not preserve the value of the sum A + B.**

T_1	T_2
read(A)	
$A := A - 50$	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
write(A)	
read(B)	
$B := B + 50$	
write(B)	
	$B := B + temp$
	write(B)

Concurrency Control & Crash Recovery

Serializability –

- **Serializability** is **related to schedules and transactions**.
- **Schedule** is a **set of transactions**, and a **transaction** is a **set of instructions** used to perform any logical operations in terms of databases.
- When **multiple transactions are running concurrently** then there is a **possibility** that the **database** may be left in an **inconsistent state**.
- **Serializability help** us to **check which schedules are serializable**.
- A **serializable schedule** is the one that always **leaves the database in consistent state.**
- **Serializability** in DBMS **ensures safe parallel execution of transactions** by enforcing rules, preventing unexpected or incorrect results.
- **Each transaction** is **treated as** if it's the **only one running, maintaining integrity**.
- **DBMS employs concurrency control technique to achieve serializability**.

Concurrency Control & Crash Recovery

Serializability –

- The **concurrency control technique helps** to ensure that **multiple transactions do not interfere with each other's execution** by controlling their access to shared resources.
- These shared resources can be database tables and records.
- By **maintaining serializability**, the **DBMS ensures** the **correctness and consistency of the database** even under heavy concurrent usage.
- **Serializability –**
  - **Serializability** is a **property of a system describing how different processes operate on shared data.**
  - A **system is serializable** if its **result is** the **same as** if the **operations** were **executed in some sequential order**.(There is **no overlap in execution**)
  - **DBMS** can be **accomplished by locking data** so that **no other process can access it** while it is being read or written.

Concurrency Control & Crash Recovery

Serializability –

- **Serializable Schedule –**
  - A **serializable schedule** is a **sequence of database actions** (read and write operations) that **does not violate the serializability property**.
  - It **ensures** that **concurrent transactions produce** the **same final result as if** they were **executed one after the other.**
  - It also **helps to maintain** the **data consistency and integrity**.
  - It is important because **it maintains the ACID property** of any Database.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - In DBMS, **serializability requires** that **transactions appear to happen in a particular order**, **even if they execute concurrently**.
  - **Transactions** that are **not serializable** may **produce incorrect results.**
  - There are **different types of serializability** in DBMS with different advantages and disadvantages.
  - The **two of the most common types**:
    - **Conflict Serializability**
    - **View Serializability**

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **Conflict Serializability –**
    - **Instructions i and j of transactions Ti and Tj** respectively, **conflict if and only if** there exists **some item Q accessed by both i and j**, and **at least one of these instructions wrote Q.**
    - 1. i = read(Q), j = read(Q). i and j don't conflict.
    - 2. i = read(Q), j = write(Q). They conflict.
    - 3. i = write(Q), j = read(Q). They conflict
    - 4. i = write(Q), j = write(Q). They conflict

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **Conflict Serializability –**
    - In this serializability, **conflicting operations on the same data items are executed in an order that preserves database consistency.**
    - **Each transaction is assigned a unique number**, and the **operations within each transaction are executed in order based on that number.**
    - It **ensures** that **no two conflicting operations are executed concurrently**.
    - For **example**, consider a **database with two tables: Customers & Orders. Each order is associated with one customer even though a single client may place orders.**

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **Conflict Serializability –**
    - **Key conditions** for conflict serializability –
      - **Both operations belong to different transactions**
      - **Both operations are on the same data item**
      - **At least one of the two operations is a write operation**
    - If two transactions were to execute concurrently, one adding an order for customer A and the other adding an order for customer B, conflict serializability would ensure that the transaction adding the order for customer A would finish before the transaction adding the order for customer B.
    - This would prevent any inconsistency in the database, such as an order being associated with the wrong customer.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **Conflict Serializability -**
    - **Examples –**
      - Conflicting operations pair (R1(A), W2(A)) because they belong to two different transactions on the same data item A and one of them is a write operation.
      - Similarly, (W1(A), W2(A)) and (W1(A),R2(A)) pairs are also conflicting.
      - On the other hand, the (R1(A), W(B)) pair is non-conflicting because they are operate on different data items.
      - Similarly, (W1(A),W2(B)) pair is non-conflicting.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **Conflict Serializability -**
    - If a **schedule S** can be **transformed into** a **schedule S´** by a **series of swaps of non-conflicting instructions**, we say that **S and S´** are **conflict equivalent.**
    - We say that a **schedule S** is **conflict serializable** if it is **conflict equivalent to a serial schedule.**

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - Let S and S´ be two schedules with the same set of transactions.
    - S and S´ are view equivalent if the following three conditions are met:
      1. For each data item Q, if transaction Ti reads the initial value of Q in schedule S, then transaction Ti must, in schedule S´, also read the initial value of Q.
      2. For each data item Q, if transaction Ti executes read(Q) in schedule S, and that value was produced by transaction Tj (if any), then transaction Ti in schedule S´ must also read the value of Q that was produced by transaction Tj .
      3. For each data item Q, the transaction (if any) that performs the final write(Q) operation in schedule S must perform the final write(Q) operation in schedule S´.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:
    - **Initial Read -** An initial read of both schedules must be the same. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

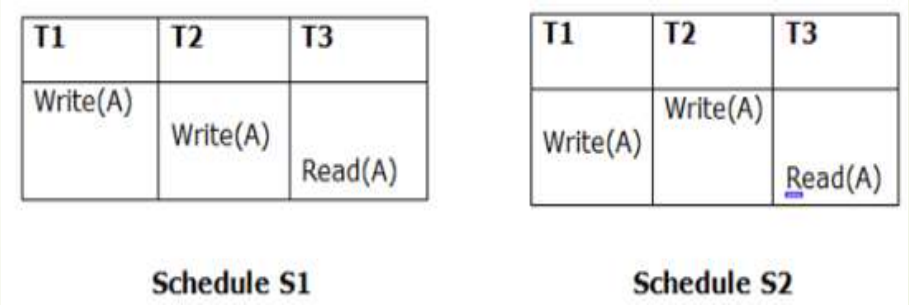
Schedule S2

    - Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - **Updated Read -** In schedule S1, if Ti is reading A which is updated by Tj then in S2 also, Ti should read A which is updated by Tj.



T1	T2	T3
Write(A)		
	Write(A)	
		Read(A)

Schedule S1

T1	T2	T3
	Write(A)	
Write(A)		
		Read(A)

Schedule S2

- Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - **Final Write -** A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)		
	Read(A)	
		Write(A)

Schedule S1

T1	T2	T3
	Read(A)	
Write(A)		
		Write(A)

Schedule S2

    - Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - View equivalence is based purely on reads and writes alone.
    - Conditions 1 and 2 ensure that each transaction reads the same values in both schedules and, therefore, performs the same computation.
    - Condition 3, coupled with conditions 1 and 2, ensures that both schedules result in the same final system state.
    - A schedule S is view serializable if it is view equivalent to a serial schedule.
    - Every conflict serializable schedule is also view serializable.
    - Some schedules which are view-serializable but not conflict serializable.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - Some **transactions perform write(Q) operations without** having performed a **read(Q) operation**. **Writes of this sort** are called **blind writes**.

T_3	T_4	T_6
read(Q)		
	write(Q)	
write(Q)		
		write(Q)

    - **Blind writes** appear **in any view-serializable schedule** that is **not conflict serializable.**

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - Example – Schedule S

T1	T2	T3
Read(A)		
	Write(A)	
Write(A)		
		Write(A)

    - With 3 transactions, the total number of possible schedule = 3! = 6
      - S1 = <T1 T2 T3>
      - S2 = <T1 T3 T2>
      - S3 = <T2 T3 T1>
      - S4 = <T2 T1 T3>
      - S5 = <T3 T1 T2>
      - S6 = <T3 T2 T1>

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - Example - Taking first schedule S1:

T1	T2	T3
Read(A) Write(A)		
	Write(A)	
		Write(A)

Schedule S1

- Step 1: Initial Read - The initial read operation in S is done by T1 and in S1, it is also done by T1.
- Step 2 : Updated Read - In both schedules S and S1, there is no read except the initial read that's why we don't need to check that condition.
- Step 3 : Final Write - The final write operation in S is done by T3 and in S1, it is also done by T3. So, S and S1 are view Equivalent.

Concurrency Control & Crash Recovery

Serializability –

- **Types of Serializability –**
  - **View Serializability -**
    - Example - The first schedule S1 satisfies all three conditions, so we don't need to check another schedule.
    - **Hence, view equivalent serial schedule is:**

      T1 → T2 → T3

Concurrency Control & Crash Recovery

Recoverability –

- **Recoverability** is a **property of database** systems that ensures that, **in the event of a failure or error**, the **system can recover the database to a consistent state.**
- **Recoverability guarantees** that **all committed transactions** are **durable** and that **their effects are permanently stored in the database**, while the **effects of uncommitted transactions** are **undone to maintain** data **consistency**.
- The recoverability property is **enforced through** the **use of transaction logs**, which **record all changes made to the database during transaction processing.**
- When a **failure occurs**, the **system uses the log to recover the database to** a **consistent state,** which **involves** either **undoing the effects of uncommitted transactions** or **redoing the effects of committed transactions.**
- **Recoverability ensures** that **data is consistent** and **durable even in the event of failures or errors.**

Concurrency Control & Crash Recovery

Recoverability –

- A **transaction may not execute completely** due to **hardware failure, system crash or software issues**. In that case, **we have to roll back the failed transaction.**
- But **some other transactions** may **also** have **used values produced by the failed transaction**. So we **have to roll back those transactions as well**.

- **Recoverable Schedule –**
  - **Schedules** in which **transactions commit only after all transactions whose changes they read commit** are called **recoverable schedules.**
  - If some **transaction Tj** is **reading value updated or written** by some **other transaction Ti**, then the **commit of Tj** must **occur after** the **commit of Ti.**

Concurrency Control & Crash Recovery

Recoverability –

- **Recoverable Schedule –**
  - **Example 1 –**

```
S1: R1(x), W1(x), R2(x), R1(y), R2(y),
          W2(x), W1(y), C1, C2;
```

  - Given schedule follows order of **Ti →Tj => C1→C2.**
  - Transaction T1 is executed before T2 hence there is no chances of conflict occur.
  - R1(x) appears before W1(x) and transaction T1 is committed before T2 i.e. completion of first transaction performed first update on data item x, hence given schedule is recoverable.

Concurrency Control & Crash Recovery

Recoverability –

- **Recoverable Schedule –**
  - **Example 2** – Consider the following schedule involving two transactions T1 and T2.

T1	T2
R(A)	
W(A)	
	W(A)
	R(A)
commit	
	commit

  - This is a recoverable schedule since T1 commits before T2, that makes the value read by T2 correct.

Concurrency Control & Crash Recovery

Recoverability –

- **Irrecoverable Schedule –**
  - When **Tj is reading the value updated by Ti** and **Tj is committed before committing of Ti**, the **schedule** will be **irrecoverable.**

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		
Failure Point				
Commit;				

Concurrency Control & Crash Recovery

Recoverability –

- **Irrecoverable Schedule –**
  - The table above shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2.
  - T2 commits. But later on, T1 fails.
  - So we have to rollback T1.
  - Since T2 has read the value written by T1, it should also be rollbacked.
  - But we have already committed that. So this schedule is irrecoverable schedule.

Concurrency Control & Crash Recovery

Recoverability –

- **Recoverable with Cascading Rollback–**
  - If **Tj is reading value updated by Ti** and **commit of Tj is delayed till commit of Ti**, the **schedule** is called **recoverable with cascading rollback.**

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
Failure Point				
Commit;				
		Commit;		

Concurrency Control & Crash Recovery

Recoverability –

- **Recoverable with Cascading Rollback–**
  - The table above shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2.
  - T1 fails. So we have to rollback T1.
  - Since T2 has read the value written by T1, it should also be rollbacked.
  - As it has not committed, we can rollback T2 as well. So it is recoverable with cascading rollback.

Concurrency Control & Crash Recovery

Recoverability –

- **Cascadeless Recoverable Rollback–**
  - If **Tj reads value updated by Ti only after Ti is committed**, the **schedule** will be **cascadeless recoverable.**

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
Commit;				
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		

Concurrency Control & Crash Recovery

Recoverability –

- **Cascadeless Recoverable Rollback–**
  - The table above shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2.
  - But if T1 fails before commit, no other transaction has read its value, so there is no need to rollback other transaction. So this is a Cascadeless recoverable schedule.

Concurrency Control & Crash Recovery

Testing for Serializability –

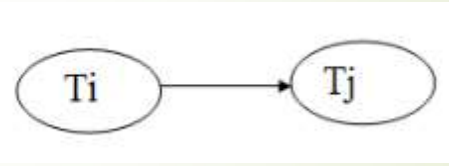
- When **designing concurrency control schemes**, we must **show that schedules generated by the scheme are serializable.**
- To do that, we must **first understand how to determine**, given a particular schedule S, **whether the schedule is serializable or not**.
- We present a simple and efficient method for determining conflict serializability of a schedule.
- Consider a schedule S. We **construct a directed graph**, called a **precedence graph, from S.**
- This **graph consists** of a **pair G=(V, E),** where **V is a set of vertices and E is a set of edges.**
- The **set of vertices** consists **of all the transactions participating in the schedule.**

Concurrency Control & Crash Recovery

Testing for Serializability –

- The **set of edges** consists of **all edges Ti →Tj** for which one of **three conditions holds:**
  - **Ti** executes **write(Q) before Tj** executes **read(Q)**.
  - **Ti** executes **read(Q) before Tj** executes **write(Q).**
  - **Ti** executes **write(Q) before Tj** executes **write(Q).**

- **Precedence Graph for Schedule S –**



  - If a **precedence graph contains a single edge Ti → Tj**, then **all the instructions of Ti** are **executed before** the **first instructions of Tj** is executed.
  - If **a precedence graph** for schedule S **contains a cycle**, then **S is non-serializable.** If the **precedence graph has no cycle**, then **S is** known as **serializable.**

Concurrency Control & Crash Recovery

Testing for Serializability –

- Precedence Graph for Schedule S –
  - Example 1 -

Time

T1	T2	T3
Read(A)		
	Read(B)	
A:= f_1(A)		
		Read(C)
	B:= f_2(B)	
	Write(B)	
		C:= f_3(C)
		Write(C)
Write(A)		
		Read(B)
	Read(A)	
	A:= f_4(A)	
Read(C)		
	Write(A)	
C:= f_5(C)		
Write(C)		
		B:= f_6(B)
		Write(B)

Schedule S1

Concurrency Control & Crash Recovery

Testing for Serializability –

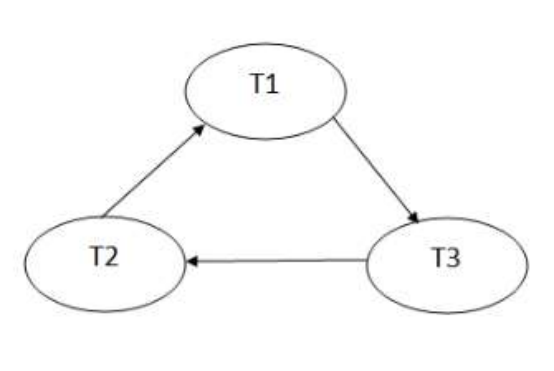
- **Precedence Graph for Schedule S –**
  - **Example 1 –**

Read(A): In T1, no subsequent writes to A, so no new edges
Read(B): In T2, no subsequent writes to B, so no new edges
Read(C): In T3, no subsequent writes to C, so no new edges
Write(B): B is subsequently read by T3, so add edge T2 → T3
Write(C): C is subsequently read by T1, so add edge T3 → T1
Write(A): A is subsequently read by T2, so add edge T1 → T2
Write(A): In T2, no subsequent reads to A, so no new edges
Write(C): In T1, no subsequent reads to C, so no new edges
Write(B): In T3, no subsequent reads to B, so no new edges

Concurrency Control & Crash Recovery

Testing for Serializability –

- **Precedence Graph for Schedule S –**
  - **Example 1** – Precedence graph for schedule S1

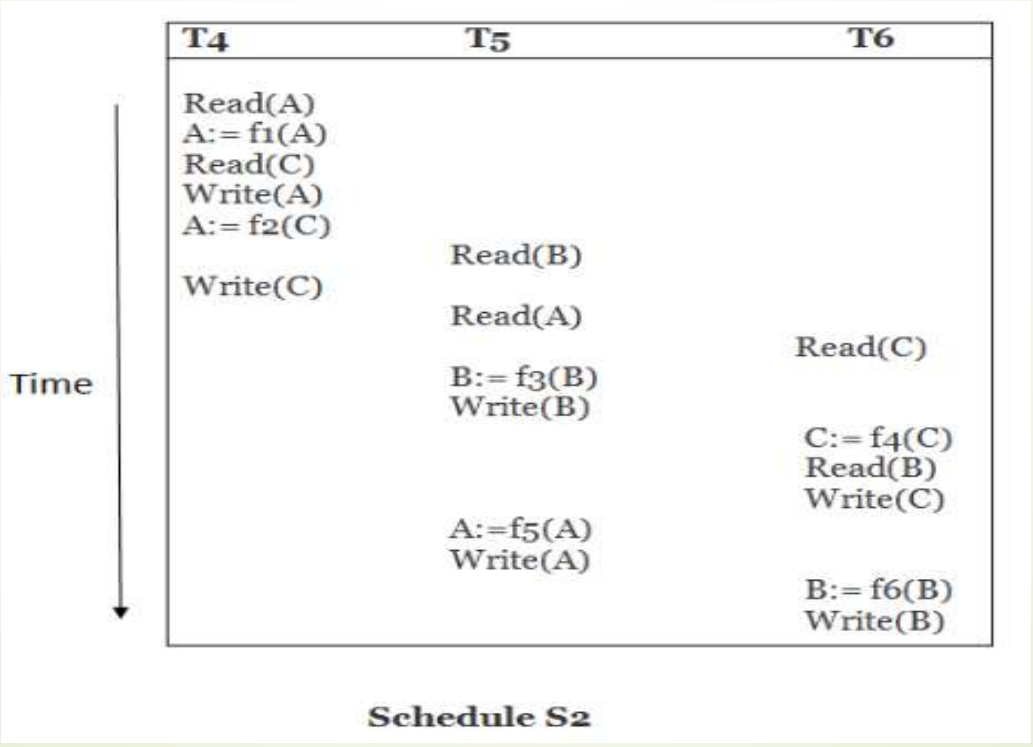


  - The precedence graph for schedule S1 contains a cycle that's why Schedule S1 is non-serializable.

Concurrency Control & Crash Recovery

Testing for Serializability –

- Precedence Graph for Schedule S –
  - Example 2 –



Schedule S2

Concurrency Control & Crash Recovery

Testing for Serializability –

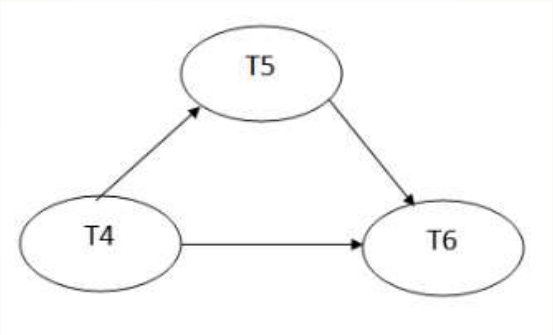
- **Precedence Graph for Schedule S –**
  - **Example 2 –**

Read(A): In T4, no subsequent writes to A, so no new edges
Read(C): In T4, no subsequent writes to C, so no new edges
Write(A): A is subsequently read by T5, so add edge T4 → T5
Read(B): In T5, no subsequent writes to B, so no new edges
Write(C): C is subsequently read by T6, so add edge T4 → T6
Write(B): A is subsequently read by T6, so add edge T5 → T6
Write(C): In T6, no subsequent reads to C, so no new edges
Write(A): In T5, no subsequent reads to A, so no new edges
Write(B): In T6, no subsequent reads to B, so no new edges

Concurrency Control & Crash Recovery

Testing for Serializability –

- **Precedence Graph for Schedule S –**
  - **Example 2** – Precedence graph for schedule S2



  - The precedence graph for schedule S2 contains no cycle that's why ScheduleS2 is serializable.

Concurrency Control & Crash Recovery

Concurrency Control –

- One of the **fundamental properties** of a **transaction is isolation**.
- When **several transactions execute concurrently** in the database then the **isolation property may no longer be preserved.**
- To ensure that, the **system** must **control** the **interaction among** the **concurrent transactions;** this control is **achieved through** one of a variety of mechanisms called **concurrency control schemes.**
- Concurrency-control schemes, **based on the serializability property**.
- There are a **variety of concurrency-control schemes**.
- **Concurrency control ensures** the **simultaneous execution or manipulation of data** by several processes or user **without resulting in data inconsistency**.
- Concurrency Control deals with **interleaved execution of more than one transaction.**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Executing a single transaction** at a time **increase** the **waiting time** of the other transactions which may result in delay in the overall execution.
- Hence for **increasing the overall throughput and efficiency** of the system, **several transactions** are **executed.**
- **Concurrency control** provides a **procedure** that is able to **control concurrent execution** of the operations in the database.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - One way **to ensure serializability** is to require that **data items** be **accessed in a mutually exclusive manner.**
  - **Mutually exclusive** means while **one transaction** is **accessing a data item**, **no other transaction** can **modify that data item**.
  - The **most common method used** to implement this requirement is to **allow a transaction to access a data item only if it is currently holding a lock on that item.**
  - A **lock** is a **mechanism to control concurrent access to a data item**.
  - Data items can be **locked in two modes :**
    - **Exclusive(X) Mode**
    - **Shared(S) Mode**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Exclusive(X) Mode:**
    - **Data item** can be both **read as well as written**.
    - X-lock is requested using **lock-X instruction**.
  - **Shared(S) Mode:**
    - Data item can **only** be **read.**
    - S-lock is requested using **lock-S instruction**.
  - **Lock requests** are **made to concurrency control manager**.
  - **Transaction** can **proceed only after request** is **granted.**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Lock Compatibility Matrix** –

	S	X
S	true	false
X	false	false

- We require that **every transaction request a lock** in an appropriate mode on data item Q, **depending on the types of operations** that it will perform on Q.
- The **use of two lock modes allows multiple transactions** to **read a data item** but **limits write access to just one transaction** at a time.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Lock Compatibility Matrix** –
    - Let A and B represent arbitrary lock modes.
    - Suppose that a **transaction Ti requests a lock of mode A on item Q on which transaction Tj (Ti != Tj) currently holds a lock of mode B.**
    - If **transaction Ti can be granted a lock on Q immediately**, in spite of the presence of the mode B lock, then we say **mode A is compatible with mode B.**
    - An element **comp(A, B)** of the **matrix has the value true** if and only if **mode A is compatible with mode B.**
    - **Shared mode** is **compatible** with **shared mode**, but **not with exclusive mode.** At any time, **several shared-mode locks can be held simultaneously** (by different transactions) on a particular data item.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Lock Compatibility Matrix** –
    - If any **transaction holds an exclusive lock on the item no other transaction may hold any lock on the item.**
    - If a **lock cannot be granted**, the **requesting transaction** is made to **wait till all incompatible locks held by other transactions** have been **released**. The lock is then granted.
    - Example of a transaction performing locking:

T_2: **lock-S**(*A*);
read (*A*);
unlock(*A*);
lock-S(*B*);
read (*B*);
unlock(*B*);
display(*A*+*B*)

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Lock Compatibility Matrix** –
    - Locking as above is not sufficient to guarantee serializability.
    - If A and B get updated in-between the read of A and B, the displayed sum would be wrong.
    - A locking protocol is a set of rules followed by all transactions while requesting and releasing locks.
    - Locking protocols restrict the set of possible schedules.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Pitfalls of Lock-Based Protocols** –
    - Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

    - Neither T3 nor T4 can make progress - executing lock-S(B) causes T4 to wait for T3 to release its lock on B, while executing lock-X(A) causes T3 to wait for T4 to release its lock on A. Such a situation is called a **deadlock.**
    - To handle a deadlock one of T3 or T4 must be rolled back and its locks released.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Pitfalls of Lock-Based Protocols** –
    - **Starvation** is also possible if concurrency control manager is badly designed.
    - Starvation occurs when a transaction is not able to get the resources it needs to proceed and is continuously delayed or blocked.
    - For example, transaction may be waiting for an X-lock on an item, while a sequence of other transactions request are granted an S-lock on the same item.
    - The **same transaction** is **repeatedly rolled back due to deadlocks.**
    - Concurrency control manager can be designed to prevent starvation.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Granting of Locks** –
    - We can **avoid starvation** of transactions by granting locks in the following manner:
      - When a **transaction Ti requests a lock** on a data item Q in a particular mode M, the **concurrency-control manager grants the lock** provided that –
        - There is **no other transaction holding a lock on Q in a mode that conflicts with M.**
        - There is **no other transaction that is waiting for a lock on Q, and made its lock request before Ti.**
    - Thus, a lock request will never get blocked by a lock request that is made later.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Two-Phase Locking Protocol** –
    - There are **two types of Locks available** – Shared & Exclusive.
    - **Implementing** this **lock system without** any **restrictions** gives us the **Simple Lock-based protocol** but they **do not guarantee Serializability.**
    - **To guarantee serializability**, we follow **additional protocols** concerning the positioning of locking and unlocking operations in every transaction.
    - We have **Two-Phase Locking(2-PL)** which **ensures serializability.**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Two-Phase Locking Protocol** –
    - A **transaction** is said to **follow the Two-Phase Locking protocol** if **Locking and Unlocking** can be **done in two phases.**
    - **Phase 1: Growing Phase**
      - transaction may **obtain locks** but may **not release locks**
    - **Phase 2: Shrinking Phase**
      - transaction may **release locks** but may **not obtain locks**
    - If lock conversion is allowed, then upgrading of lock(from S(a) to X(a)) is allowed in the Growing Phase, and downgrading of lock (from X(a) to S(a)) must be done in the shrinking phase.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Two-Phase Locking Protocol** –
    - **Two-phase locking with lock conversions:**
      - **Growing Phase / First Phase**:
        - can acquire a lock-S on item
        - can acquire a lock-X on item
        - can convert a lock-S to a lock-X (upgrade)
      - **Shrinking Phase / Second Phase:**
        - can release a lock-S
        - can release a lock-X
        - can convert a lock-X to a lock-S (downgrade)

Concurrency Control & Crash Recovery

Concurrency Control –

- **Lock Based Protocol –**
  - **Two-Phase Locking Protocol** –
    - Two-phase locking **does not ensure freedom from deadlocks.**
    - **Cascading roll-back** is **possible under two phase locking**.
    - **To avoid** this, follow a **modified protocol** called **strict two-phase locking**.
    - Here a **transaction** must **hold all its exclusive locks till it commits/aborts**.
    - **Rigorous two-phase locking** is even **stricter**: here **all locks are held till commit/abort.**
    - In this protocol, **transactions can be serialized in the order in which they commit.**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Timestamp-Based Protocols –**
  - **Each transaction** is **issued** a **timestamp** when it **enters the system.**
  - If an **old transaction Ti** has **time-stamp TS(Ti),** a **new transaction Tj** is **assigned time-stamp TS(Tj)** such that **TS(Ti) <TS(Tj).**
  - The **protocol manages concurrent execution** such that the **time-stamps determine the serializability order.**
  - There are **two simple methods for implementing** this scheme:
    - **Use** the **value of the system clock as** the **timestamp.**
    - **Use a logical counter** that is **incremented after a new timestamp** has been assigned.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Timestamp-Based Protocols –**
  - In order to assure such behavior, the **protocol maintains for each data Q two timestamp values**:
    - **W-timestamp(Q)** is the **largest time-stamp** of any transaction that **executed write(Q) successfully**.
    - **R-timestamp(Q)** is the **largest time-stamp** of any transaction that **executed read(Q) successfully.**
  - The timestamp-based protocol **ensures** that any **conflicting read and write operations** are **executed in timestamp order**.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Timestamp-Based Protocols –**
  - Suppose a **transaction Ti issues a read(Q) operation -**
    - If **TS(Ti) ≤ W_TS(Q),** then **Ti needs to read a value of Q** that was **already overwritten**. Hence, the **read operation is rejected**, and **Ti is rolled back.**
    - If **TS(Ti) ≥ W_TS(Q)**, then the **read operation is executed**, and **R_TS(Q) is set to the maximum of R_TS(Q) and TS(Ti).**
    - **Where,**
      - TS(Ti) denotes the timestamp of the transaction Ti.
      - R_TS(Q) denotes the Read time-stamp of data-item Q.
      - W_TS(Q) denotes the Write time-stamp of data-item Q.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Timestamp-Based Protocols –**
  - Suppose a transaction **Ti issues a write(Q) operation**
    - If **TS(Ti) < R_TS(Q)**, then the value of Q that Ti is producing was needed previously, and system assumed that value would never be produced. Hence, the **write operation is rejected**, and **Ti is rolled back**.
    - If **TS(Ti) < W_TS(Q)**, then Ti is attempting to write an obsolete value of Q. Hence, this **write operation is rejected**, and **Ti is rolled back.**
    - **Otherwise**, the **write operation is executed**, and **W_TS (Q) is set to TS(Ti).**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Validation Based Protocol –**
  - **Validation Based Protocol** is also called **Optimistic Concurrency Control Technique.**
  - This protocol is used in DBMS for **avoiding concurrency in transactions**.
  - It is called **optimistic because of the assumption** it makes, i.e. **very less interference occurs**, therefore, there is **no need for checking while the transaction is executed.**
  - In this technique, **no checking is done while the transaction is been executed.**
  - **Until the transaction end is reached, updates in** the **transaction are not applied directly to the database.**
  - **All updates** are **applied to local copies of data items** kept for the transaction.

Concurrency Control & Crash Recovery

Concurrency Control –

- **Validation Based Protocol –**
  - **At the end of transaction execution**, a **validation phase checks** whether any of **transaction updates violate serializability**.
  - If there is **no violation of serializability** the **transaction is committed** and the **database is updated**; or **else**, the **transaction is updated and then restarted.**
  - **Each transaction** Ti **executes in three different phases** in its lifetime, depending on whether it is a read-only or an update transaction.
    - **Read Phase**
    - **Validation Phase**
    - **Write Phase**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Validation Based Protocol –**
  - **Read Phase –**
    - During this phase, the **system executes transaction Ti.**
    - It **reads the values** of the various data items and **stores them in variables local to Ti.**
    - It **performs all write operations on temporary local variables**, **without updates to the actual database.**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Validation Based Protocol –**
  - **Validation Phase –**
    - Transaction Ti performs a **validation test to determine** whether it can **copy to the database** the **temporary local variables** that **hold the results of write operations without causing a violation of serializability**.
  - **Write Phase –**
    - If transaction **Ti succeeds in validation** (step 2), then the **system applies the actual updates to the database.**
    - **Otherwise, the system rolls back Ti.**

Concurrency Control & Crash Recovery

Concurrency Control –

- **Validation Based Protocol –**
  - Each **transaction Ti has 3 timestamps** –
    - **Start(Ti)** : It is the time when Ti started its execution.
    - **Validation(Ti)**: It is the time when Ti just finished its read phase and begin its validation phase.
    - **Finish(Ti)** : The time when Ti end it's all writing operations in the database under write-phase.
  - **Serializability order** is **determined by timestamp** given at validation time, to **increase concurrency**.
  - This protocol is useful and gives **greater degree of concurrency** if probability of **conflicts is low.**

Concurrency Control & Crash Recovery

Data Recovery –

- **A computer system**, like any other device, is **subject to failure from a variety of causes:**
- For e.g. **disk crash, power outage, software error, a fire in the machine room**.
- **In any failure, information may be lost.**
- Therefore, the **database system** must **take actions in advance** to **ensure that** the **atomicity and durability properties of transactions are preserved**.
- An **integral part of a database system** is a **recovery scheme** that can **restore the database to the consistent state** that **existed before the failure.**
- The **recovery scheme** must **also provide high availability.**
- It **must minimize** the **time for which** the **database is not usable after a crash.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Failure Classification –**
  - There are **various types of failure** that may **occur in a system**, **each** of which **needs to be dealt with in a different manner**.
  - The **simplest type of failure** is one **that does not result in the loss of information** in the system.
  - The **failures that are more difficult to deal** with are **those that result in loss of information.**
  - **1. Transaction failure:**
    - There are **two types of errors** that may **cause a transaction to fail**:
    - **Logical error** - The **transaction can no longer continue** with its normal execution because of **some internal condition**, such as **bad input**, **data not found**, **overflow, or resource limit exceeded.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Failure Classification –**
  - **1. Transaction failure:**
    - **System error -** The **system has entered** an **undesirable state** (for example, deadlock), as a **result** of which a **transaction cannot continue** with its **normal execution.** The **transaction can be re-executed** at a later time.
  - **2. System Crash:**
    - There is a **hardware malfunction**, or a **bug in the database software or** the **operating system**, that **causes the loss of the content of volatile storage**, and **brings transaction processing to a halt**.
    - The **content of non-volatile storage remains intact**, and **is not corrupted**.
    - The **assumption that hardware errors and bugs in the software bring** the **system to a halt**, but **do not corrupt the nonvolatile storage contents**, is **known as** the **fail-stop assumption.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Failure Classification –**
  - **2. System Crash:**
    - **Well-designed systems have** numerous **internal checks**, **at the hardware and the software level**, that **bring the system to a halt when there is an error.**
    - Hence, the **fail-stop assumption is a reasonable one**.
  - **3. Disk failure:**
    - A **disk block loses its content** as a **result of** either a **head crash** or **failure during a data-transfer operation.**
    - **Copies of the data on other disks**, or archival backups on tertiary media, such as DVD or tapes, are **used to recover from the failure.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Failure Classification –**
  - To **determine how the system should recover from failures**, we **need to identify the failure modes of those devices used for storing data.**
  - Next, we must **consider how these failure modes affect the contents of the database.**
  - We can **then propose algorithms to ensure database consistency and transaction atomicity despite failures.**
  - These **algorithms**, **known as recovery algorithms**, have **two parts**:
    - **Actions taken during normal transaction processing** to **ensure** that **enough information exists to allow recovery from failures.**
    - **Actions taken after a failure to recover the database contents to a state** that **ensures database consistency, transaction atomicity, and durability**.

Concurrency Control & Crash Recovery

Data Recovery –

- **Recovery and Atomicity –**
  - **Modifying** the **database without ensuring** that the **transaction will commit** may **leave the database in** an **inconsistent state.**
  - Consider **transaction Ti** that **transfers $50 from account A to account B**; **goal** is **either** to **perform all database modifications** made by Ti or **none at all.**
  - Several **output operations** may be **required for Ti** (to output A and B). A **failure** may **occur after one of these modifications** have been **made but before all of them are made.**
  - To **ensure atomicity** despite failures, we **first output information describing the modifications to stable storage without modifying the database itself.**
  - **Two approaches – log-based recovery and shadow paging.**
  - We **assume** (initially) that **transactions run serially**, that is, **one after the other.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - A **log** is **kept on stable storage.**
  - The **log** is a **sequence of log records**, and **maintains a record of update activities on the database.**
  - When **transaction Ti starts**, it **registers itself** by writing a **<Ti start> log record.**
  - **Before Ti executes write(X)**, a **log record <Ti, Xj, V1, V2> is written**, where **V1 is the value of X before the write**, and **V2 is the value to be written to X**.
  - Log record notes that Ti has performed a write on data item Xj. The Xj had value V1 before the write, and will have value V2 after the write.
  - When **Ti finishes its last statement**, the **log record <Ti commit> is written.**
  - We **assume** for now that **log records are written directly to stable storage** (that is, they are not buffered).

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Two approaches using logs –**
    - Deferred database modification
    - Immediate database modification
  - **Deferred database modification –**
    - This scheme **records all modifications to the log**, but **defers all the writes after partial commit.**
    - Assume that **transactions execute serially**, **Transaction starts by writing <Ti start> record to log.**
    - A **write(X) operation results in a log record <Ti, X, V>** being written, where **V is the new value for X.** The **old value is not needed for this scheme.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Deferred database modification –**
    - The **write is not performed on X** at this time, but is **deferred.**
    - When **Ti partially commits**, **<Ti commit> is written to the log.**
    - Finally, the **log records are read and used to execute** the **previously deferred writes.**
    - **During recovery** after a crash, a **transaction needs to be redone if and only if** both **<Ti start> and <Ti commit>** are there **in the log.**
    - **Redoing a transaction Ti** (redo Ti) **sets the value of all data items updated by the transaction to the new values.**

Concurrency Control & Crash Recovery

Data Recovery –

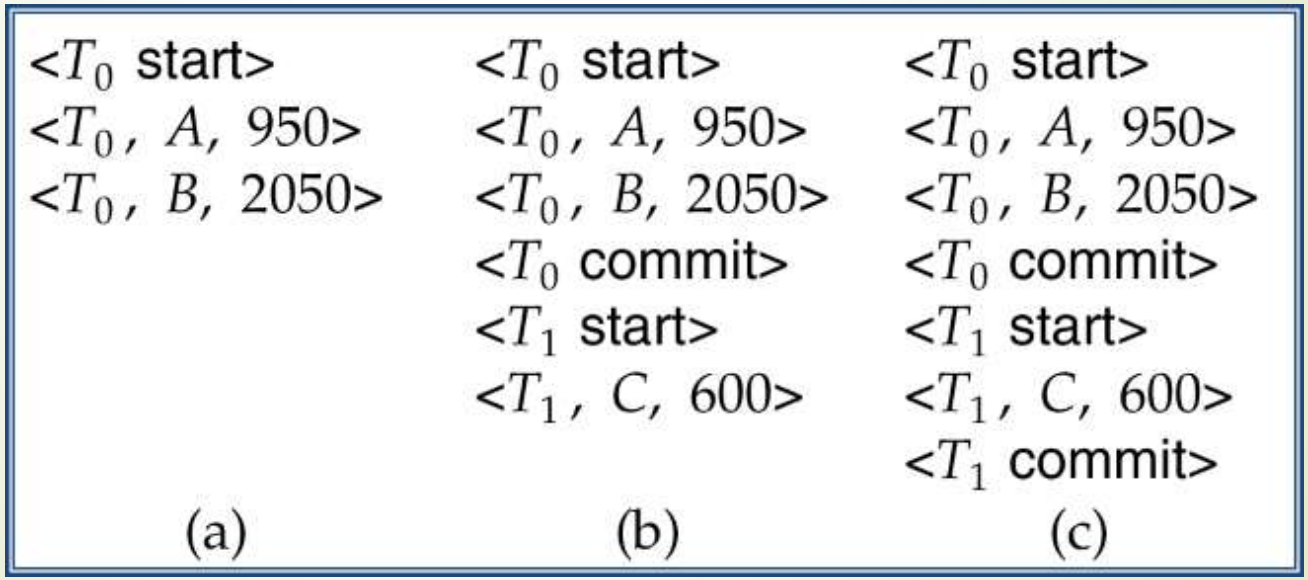
- **Log-Based Recovery –**
  - **Deferred database modification –**
    - **Crashes can occur** while –
      - **Transaction is executing the original updates**, or
      - While **recovery action is being taken**
    - **Example** - transactions T0 and T1 (T0 executes before T1):

T_0: **read** (A)	T_1 : **read** (C)
A: - A - 50	*C:-C- 100*
Write (A)	**write** (C)
read (B)	
B:- B + 50	
write (B)	

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Deferred database modification –**
    - Below we show the log as it appears at three instances of time.



Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Deferred database modification –**
    - If **log on stable storage at time of crash** is as in case:
      - **a) No redo** actions need to be taken
      - **b) redo(T0)** must be performed since <T0 commit> is present
      - **c) redo(T0)** must be performed **followed by redo(T1)** since <T0 commit> and <Ti commit> are present

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Immediate database modification –**
    - This scheme **allows database updates of an uncommitted transaction** to be **made as the writes are issued**.
    - Since **undoing may be needed**, **update logs must have both old value and new value.**
    - **Update log record** must be **written before database item is written.**
    - **Output of updated blocks** can **take place at any time before or after transaction commit.**
    - **Order in which blocks are output** can be **different from** the **order in which they are written.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Immediate database modification –**

Log	Write	Output
<T_0 **start**>		
<T_0, A, 1000, 950>		
T_0, B, 2000, 2050		
	$A = 950$	
	$B = 2050$	
<T_0 **commit**>		
<T_1 **start**> x_1		
<T_1, C, 700, 600>		
	$C = 600$	
		B_B, B_C
<T_1 **commit**>		
		B_A

- Note: B_X denotes block containing X.

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Immediate database modification –**
    - **Recovery procedure** has **two operations** instead of one:
      - **undo(Ti) restores** the **value of all data items updated by Ti** to their **old values**, going **backwards** from the last log record for Ti
      - **redo(Ti) sets** the **value of all data items** updated by Ti to the **new values**, going **forward** from the first log record for Ti
    - When **recovering after failure:**
      - **Transaction Ti** needs to be **undone** if the **log contains the record <Ti start>**, but **does not contain the record <Ti commit>.**
      - **Transaction Ti** needs to be **redone** if the **log contains both the record <Ti start> and the record <Ti commit>.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Log-Based Recovery –**
  - **Immediate database modification –**
    - Below we show the log as it appears at three instances of time.

(a)	(b)	(c)
$<T_0$ start>	$<T_0$ start>	$<T_0$ start>
$<T_0, A, 1000, 950>$	$<T_0, A, 1000, 950>$	$<T_0, A, 1000, 950>$
$<T_0, B, 2000, 2050>$	$<T_0, B, 2000, 2050>$	$<T_0, B, 2000, 2050>$
	$<T_0$ commit>	$<T_0$ commit>
	$<T_1$ start>	$<T_1$ start>
	$<T_1, C, 700, 600>$	$<T_1, C, 700, 600>$
		$<T_1$ commit>

Recovery actions in each case above are:

(a) undo (T_0): B is restored to 2000 and A to 1000.

(b) undo (T_1) and redo (T_0): C is restored to 700, and then *A* and *B* are set to 950 and 2050 respectively.

(c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then *C* is set to 600

Concurrency Control & Crash Recovery

Data Recovery –

- **Checkpoints –**
  - **Problems in recovery procedure –**
    - **searching** the **entire log is time-consuming**
    - We might **unnecessarily redo transactions** which have **already output their updates to the database.**
  - **Streamline recovery procedure by periodically performing checkpointing**.
    - **Output all log records currently residing in main memory onto stable storage.**
    - **Output all modified buffer blocks to the disk.**
    - **Write a log record < checkpoint> onto stable storage.**

Concurrency Control & Crash Recovery

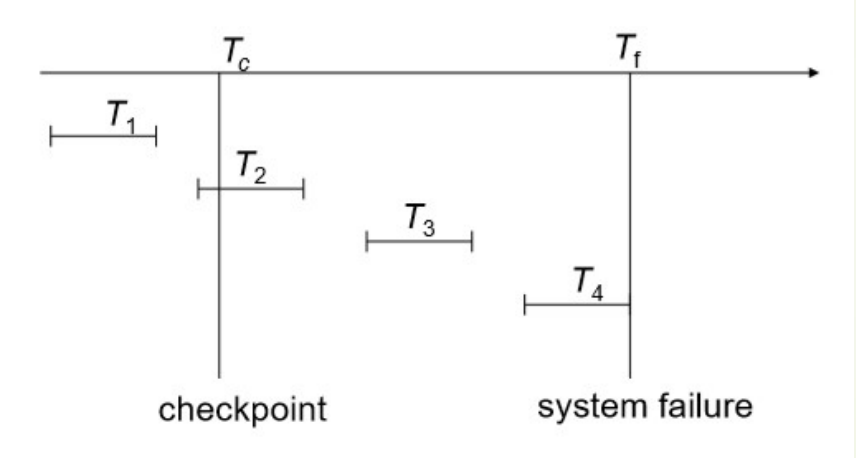
Data Recovery –

- **Checkpoints –**
  - **During recovery** we need to consider only the **most recent transaction Ti that started before the checkpoint**, and **transactions that started after Ti.**
    - **Scan backwards** from **end of log to find the most recent <checkpoint> record**
    - **Continue scanning backwards** till a **record <Ti start> is found**.
    - Need **only consider** the **part of log following above start record**. **Earlier part of log** can be **ignored** during recovery, and can be **erased whenever desired.**
    - **For all transactions** (starting from Ti or later) **with no <Ti commit>, execute undo(Ti)**. (Done only in case of immediate modification.)
    - **Scanning forward** in the log, for **all transactions starting from Ti or later with a <Ti commit>, execute redo(Ti).**

Concurrency Control & Crash Recovery

Data Recovery –

- **Checkpoints –**
  - **Example of Checkpoints –**



- T1 can be ignored (updates already output to disk due to checkpoint)
- T2 and T3 redone.
- T4 undone

Concurrency Control & Crash Recovery

Data Recovery –

- **Shadow Paging –**
  - **Shadow paging** is an **alternative to log-based recovery.**
  - This scheme is **useful if transactions execute serially .**
  - **Idea: maintain two page tables during the lifetime of a transaction – the current page table, and the shadow page table.**
  - **Store** the **shadow page table in nonvolatile storage**, such that **state of the database prior to transaction execution** may be **recovered.**
  - **Shadow page table** is **never modified during execution.**
  - To start with, **both the page tables are identical**.
  - **Only current page table is used for data item accesses during execution of the transaction.**

Concurrency Control & Crash Recovery

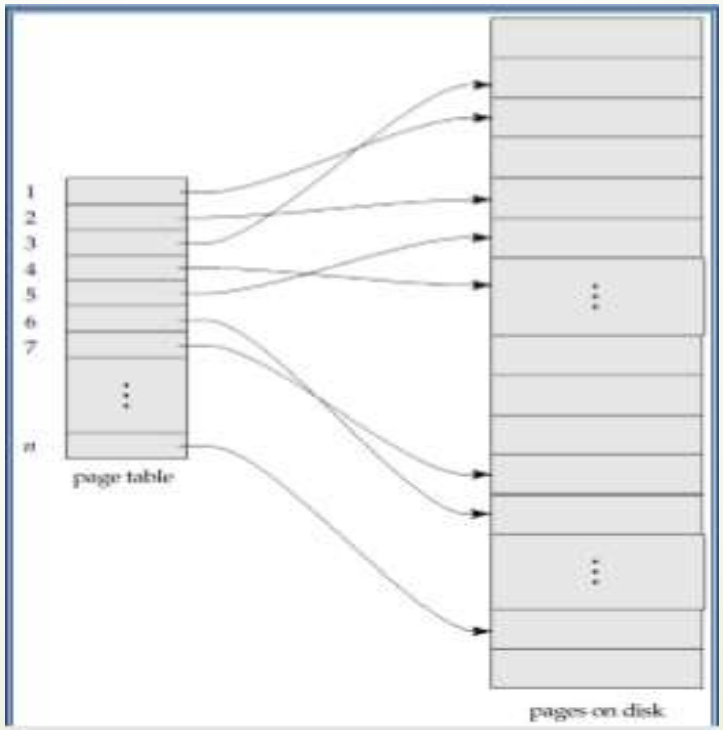
Data Recovery –

- **Shadow Paging –**
  - Whenever **any page** is **about to be written for the first time-**
    - A **copy of this page is made onto an unused page**.
    - The **current page table** is then **made to point to the copy.**
    - The **update** is **performed on the copy.**

Concurrency Control & Crash Recovery

Data Recovery –

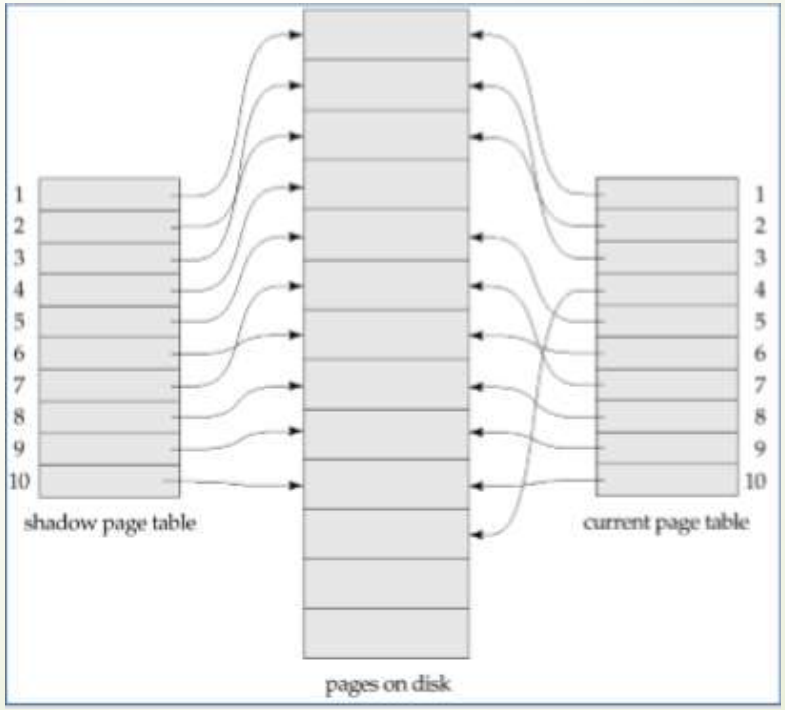
- Shadow Paging –
  - Sample Page Table



Concurrency Control & Crash Recovery

Data Recovery –

- **Shadow Paging –**
  - **Example** - Shadow and current page tables after write to page 4



Concurrency Control & Crash Recovery

Data Recovery –

- **Shadow Paging –**
  - **To commit a transaction** :
    - **Flush all modified pages in main memory to disk**
    - **Output current page table to disk**
    - **Make** the **current page table** the **new shadow page table**, as follows:
      - **keep a pointer to the shadow page table at a fixed (known) location on disk.**
      - **to make the current page table the new shadow page table**, simply **update the pointer to point to current page table on disk**
    - **Once pointer to shadow page table has been written, transaction is committed.**

Concurrency Control & Crash Recovery

Data Recovery –

- **Shadow Paging –**
  - **To commit a transaction** :
    - **No recovery is needed after a crash** — new transactions can start right away, using the shadow page table.
    - **Pages not pointed to from current/shadow page table should be freed** (garbage collected).
  - **Advantages** of shadow-paging **over log-based schemes** –
    - **no overhead of writing log records**
    - **recovery is trivial**

Concurrency Control & Crash Recovery

Data Recovery –

- **Shadow Paging –**
  - **Disadvantages** –
    - **Copying** the **entire page table** is **very expensive**
    - **Commit overhead is high** even with above extension
    - **Data gets fragmented** (related pages get separated on disk)
    - **After every transaction completion**, the **database pages containing old versions of modified data need to be garbage collected**
    - **Hard to extend algorithm** to **allow transactions to run concurrently**